

Taller Práctico: Mini Pokédex con Next.js e IA

Delegar la escritura de código a un asistente de IA sin un marco de trabajo sólido es una receta para el caos técnico. La verdadera potencia de herramientas como **Codex** o **Gemini** no reside en generar sintaxis, sino en cómo el profesional orquesta un ecosistema de información que guíe al modelo hacia resultados precisos y escalables. Este enfoque, basado en archivos de definición de agentes, especificaciones de API y reglas de control de versiones, transforma una simple petición en una ejecución de ingeniería de software coherente. El éxito del proyecto depende de la infraestructura de conocimiento que rodea al repositorio, permitiendo que la IA actúe como un **desarrollador experto integrado**.

La integración de activos como la documentación OpenAPI y el archivo `agents.md` establece la soberanía técnica necesaria. Al predefinir el uso de **TypeScript**, el versionado semántico en el `package.json` y el registro en un **Changelog**, se elimina la ambigüedad que deriva en errores. Este método garantiza que cada iteración mantenga una **trazabilidad total**. La orquestación de estas herramientas reduce los tiempos de desarrollo y asegura que el producto final cumpla con los estándares de calidad exigidos en entornos corporativos.

Configuración del Ecosistema y Definición de Reglas

Establecimiento del Repositorio y Entorno Base

El proceso comienza con la inicialización de un repositorio en GitHub y la configuración de un proyecto **Next.js** limpio. Es fundamental limpiar la estructura inicial (*boilerplate*) y centralizar los recursos. El uso de gestores de paquetes modernos como **pnpm** asegura una instalación de dependencias eficiente y predecible para el posterior despliegue en plataformas como Vercel o Netlify.

Creación del Archivo de Agente (`agents.md`)

Para que el asistente comprenda su rol, se define un documento maestro en la raíz que especifica:

- ****Roles:**** Actuar como Desarrollador Senior en Next.js y especialista en PokeAPI.
- ****Restricciones:**** Uso exclusivo de TypeScript, Tailwind CSS y principios *Clean Code*.
- ****Automatización:**** Instrucción para incrementar la versión del proyecto y documentar cada cambio en el `CHANGELOG.md` siguiendo el estándar de *Keep a Changelog*.

Inyección de Contexto y Documentación de API

Para evitar errores de conexión (404 o tipos incorrectos), proporcionamos a la IA la estructura exacta de los datos. Ante la falta de un OpenAPI oficial de PokeAPI, generamos una especificación JSON personalizada basada en la documentación. Este archivo se deposita en la carpeta `/context`, sirviendo como 'fuente de verdad' única para que la IA realice las llamadas a los *endpoints* con total precisión técnica.

Iteración y Refinamiento del Desarrollo

Generación de Código y Gestión de Pull Requests

El desarrollo se ejecuta mediante prompts simplificados que remiten al contexto ya cargado. El flujo de trabajo incluye: 1. **Feature Branches:** La IA genera ramas específicas para cada funcionalidad. 2. **Code Review:** Verificación local mediante `git checkout` antes del merge. 3. **Hotfixing:** Resolución de errores (ej. configuración de dominios de imágenes en `next.config.js`) inyectando el error directamente en el chat para corrección automática.

Evolución Funcional del Proyecto

Tras establecer la base, se implementan mejoras como el filtrado por tipo o el motor de búsqueda en tiempo real. Cada acción dispara la actualización del **versionado semántico**, garantizando un historial de *commits* profesional e impecable.

Observaciones Clave

- Prompt Chaining: Es preferible encadenar instrucciones cortas para que la IA valide la estructura antes de escribir cientos de líneas.
- Auditoría TypeScript: Revisar que la IA no degrade el proyecto a JavaScript; la tipado fuerte es innegociable para la mantenibilidad.
- Imágenes Externas: Next.js requiere autorizar dominios externos (`raw.githubusercontent.com`) en la configuración para renderizar los *sprites* de los Pokémon.
- Documentación: El uso de extensiones de previsualización Markdown en **VS Code** facilita la auditoría de los documentos autogenerados.

Conclusión

La construcción de esta Pokédex demuestra que el desarrollo moderno trata sobre la **gestión estratégica del contexto**. Al proporcionar reglas claras y datos estructurados, el profesional se eleva al rol de arquitecto. Dominar la interacción entre repositorios, documentación automatizada y modelos de lenguaje es la competencia que permite integrar la IA en procesos de negocio con resultados predecibles, seguros y de alto valor comercial.